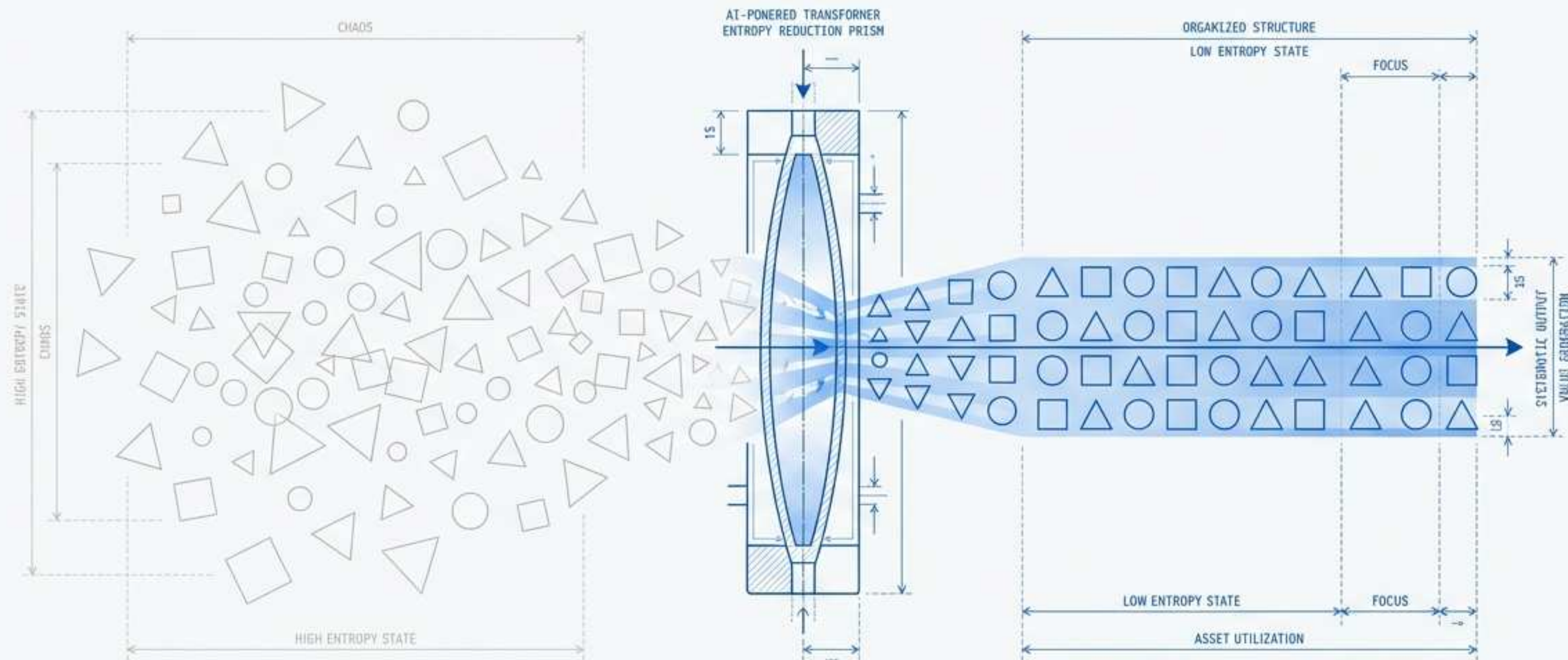


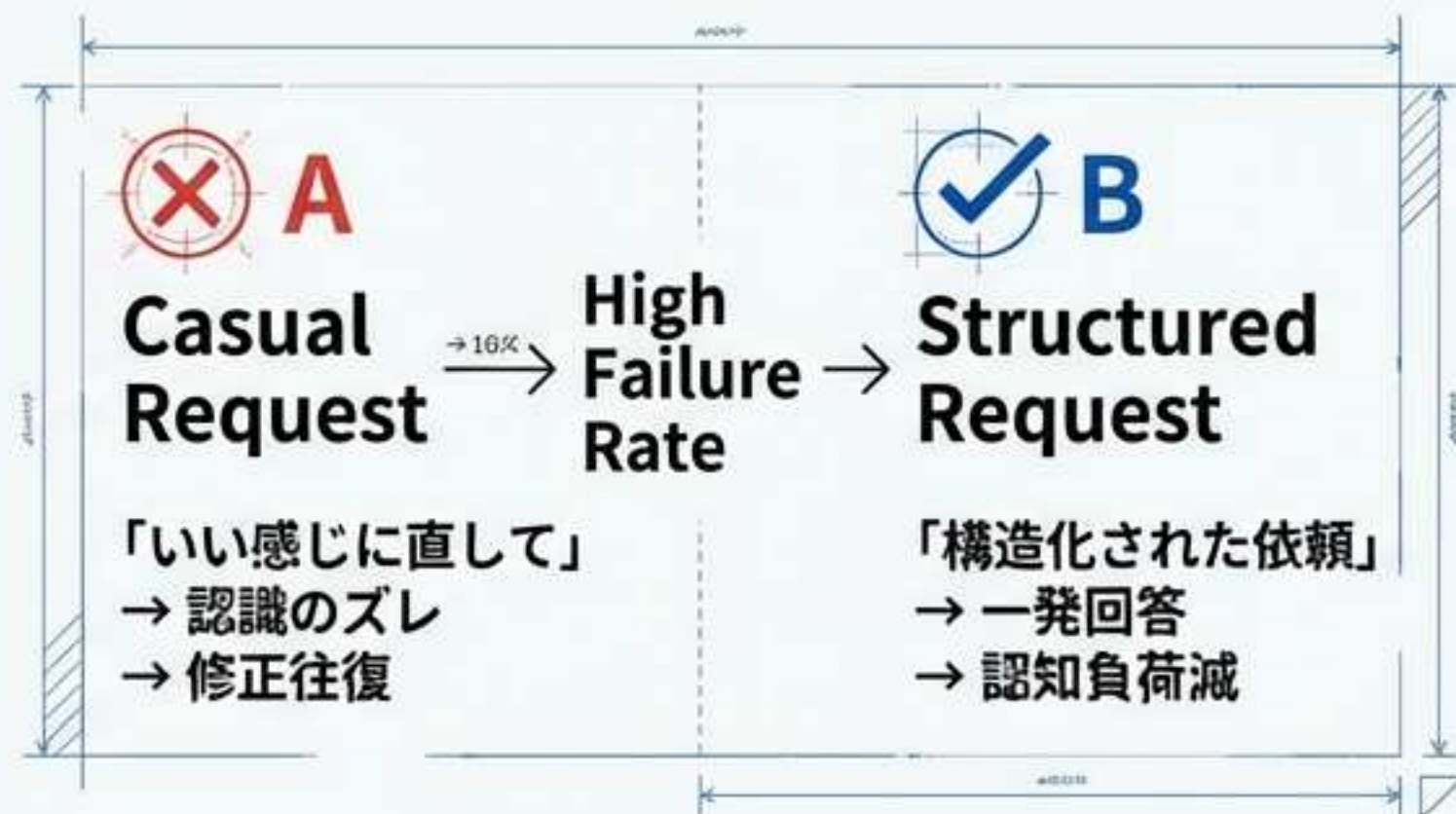
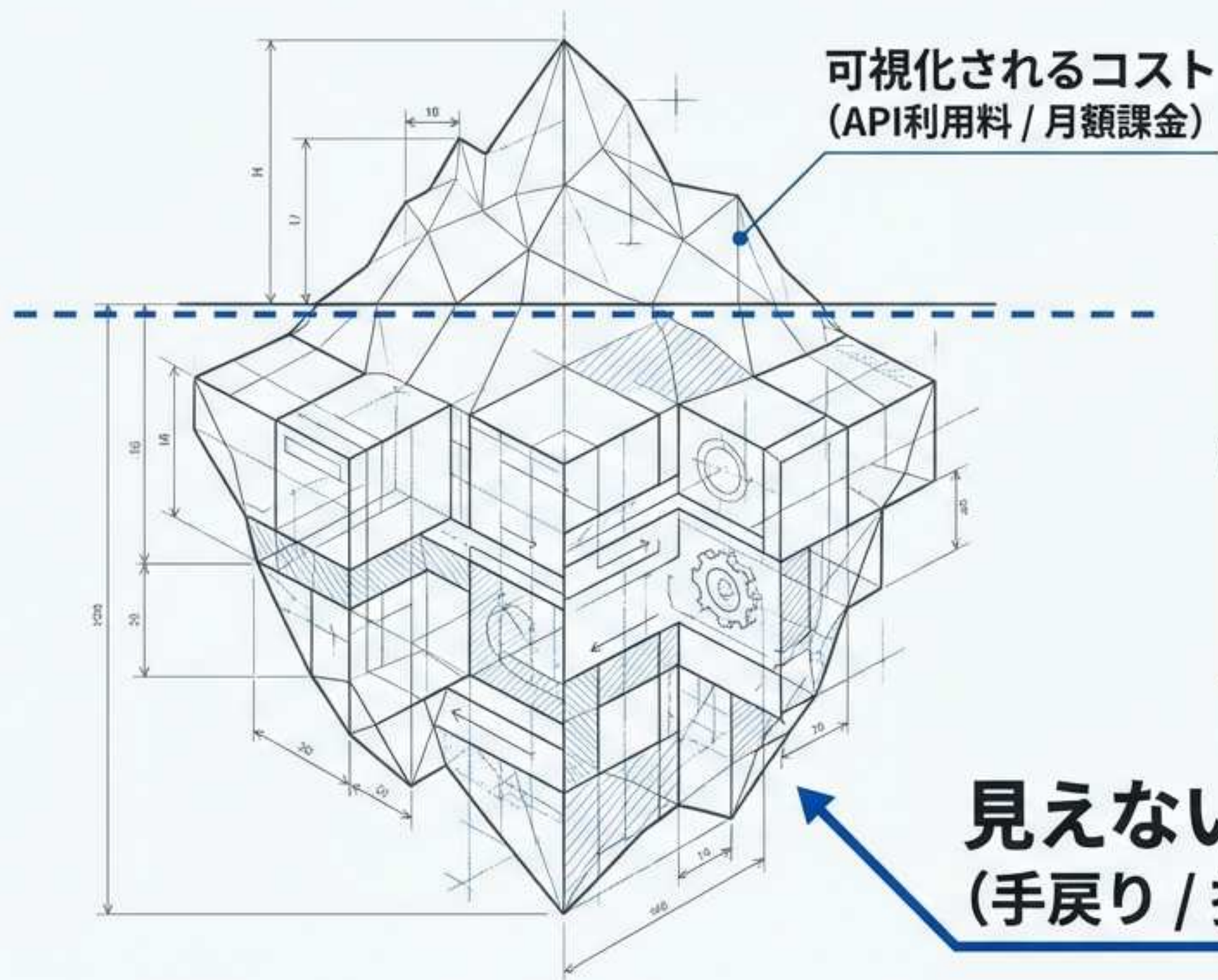


AI活用のコスパを最大化する技術

トークン制約時代における依頼設計・CLI運用・チーム資産化

Designed for Engineering Teams & Tech Leads

曖昧な依頼は「手戻り」という見えないコストを生む



コスパの定義を「単価」から「完了までの総コスト」へ

$$\text{Total Cost} = (\text{Prep} + \text{Gen} + \text{Rework}) \times \text{Freq}$$



真の効率化とは、トークン単価を気にすることではなく、この「Rework」をゼロに近づける設計にある。

回避すべき3つの失敗パターン

×

The One-Liner
(一行依頼)

「これ、いい感じに
修正して」



優先順位が不明で、
出力がブレる。

×

No Constraints
(制約なし)

(文字数やフォーマット
の指定がない)

形式修正のための追加
プロンプトが必要になる。

×

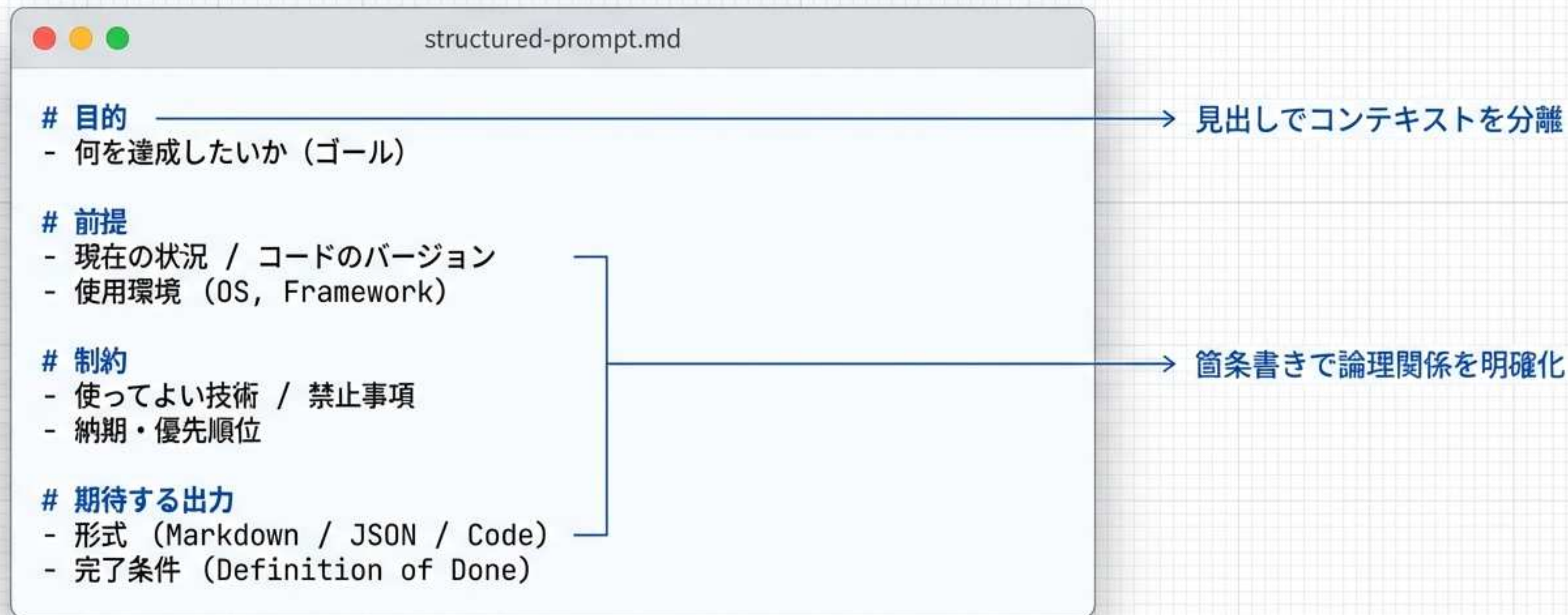
Big Bang
(一括丸投げ)



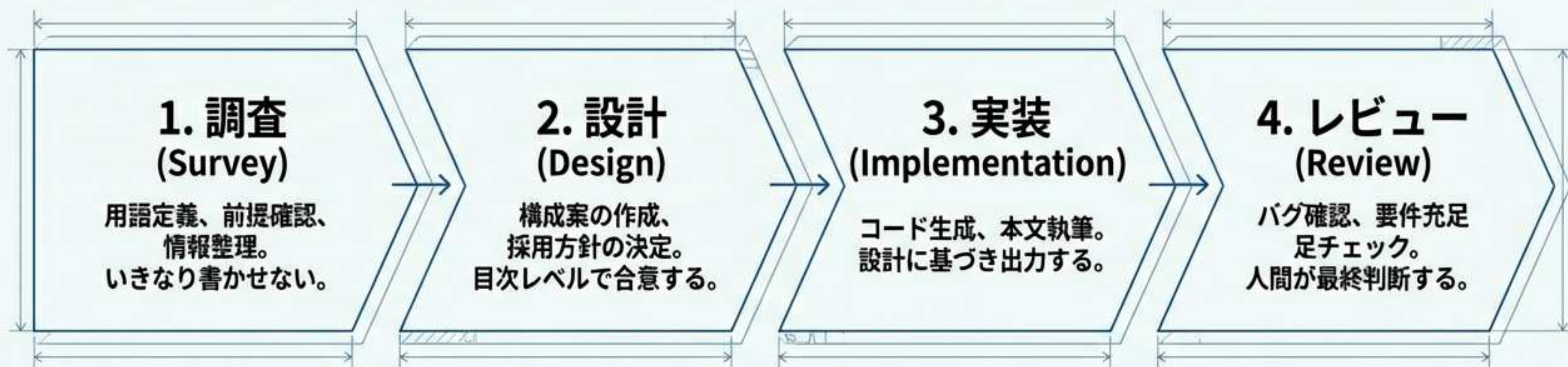
複雑すぎて推論精度が
著しく低下する。

解決策：Markdownによる構造化プロンプト

AIは自然言語よりも「構造」を理解する。Markdownで見出しと箇条書きを使うだけで精度は上がる。



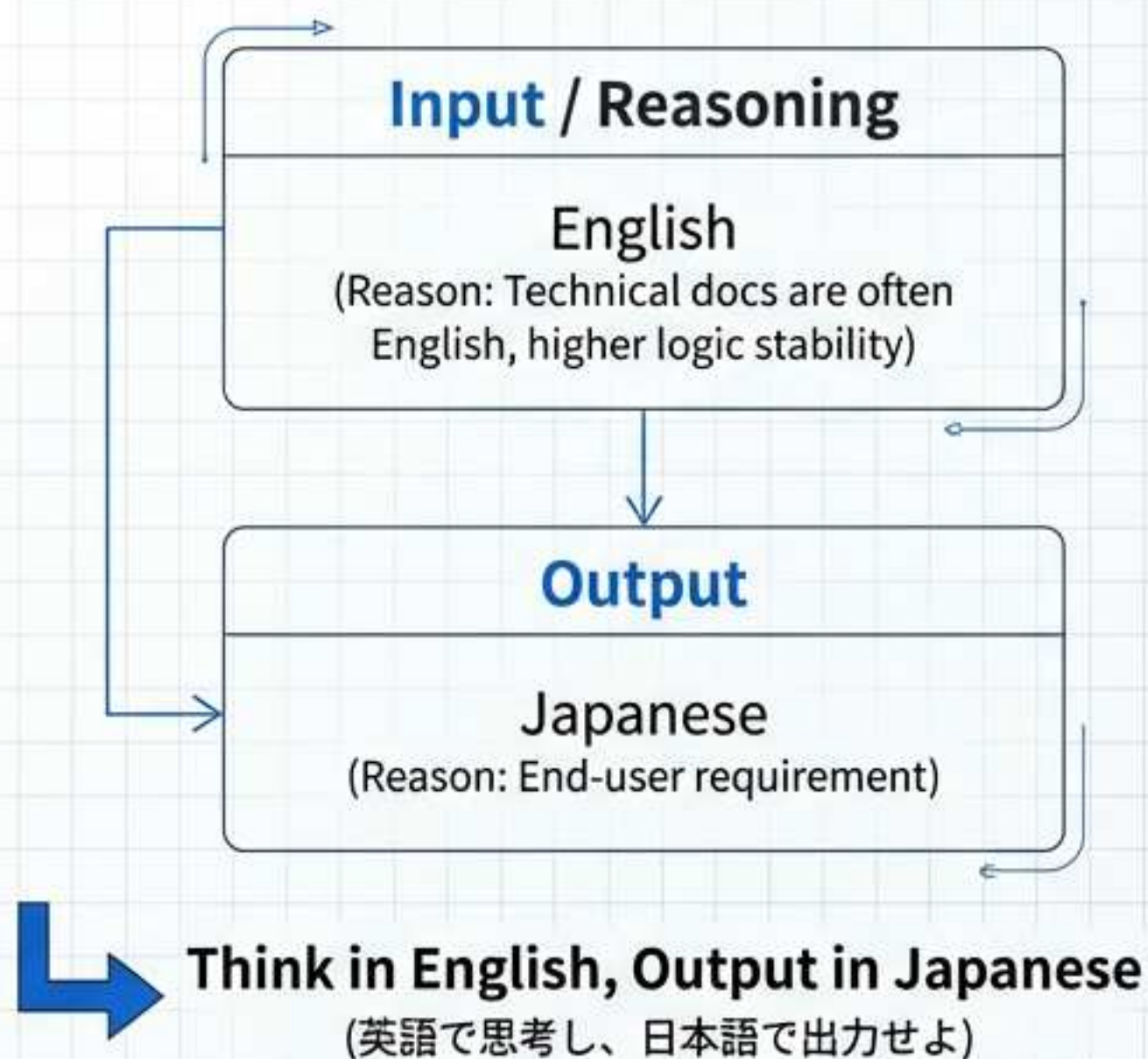
複雑なタスクは「4フェーズ」に分割して精度を上げる



各フェーズで一度AIを止めることで、手戻りの原因箇所を特定しやすくなる。

言語戦略と品質担保チェックリスト

Language Matrix



Quality Assurance Checklist

The diagram shows a clipboard with a blue clip at the top. The clipboard contains a checklist with four items, each preceded by a checkbox. The items are in Japanese. The clipboard is surrounded by dimension lines indicating its size.

- ☐ ターゲット読者は定義されているか？
- ☐ 「完了条件 (Definition of Done)」は明確か？
- ☐ 出力フォーマット (JSON/Markdown) は指定したか？
- ☐ 「やってはいけないこと (禁止事項)」は含めたか？

GUIからCLIへ：プロフェッショナルのための環境移行

 GUI (Web / Chat)	 CLI (Terminal / API)
Use Case One-off questions	Use Case Engineering Workflow
Cons × 再現性が低い × 履歴が追いつらい × 作業の邪魔になる	Pros  Performance: 応答速度と処理効率  Reproducibility: コマンドとして履歴に残る  Efficiency: 不要なトークン消費を抑制

AIは「対話相手」から「コマンドラインツール」へ。

再現性と履歴管理による「運用コスト」の削減

Non-intrusive

IDEのエディタ画面を専有せず、
バックグラウンドで処理。

History as Asset

コマンド履歴がそのまま
ナレッジになる。

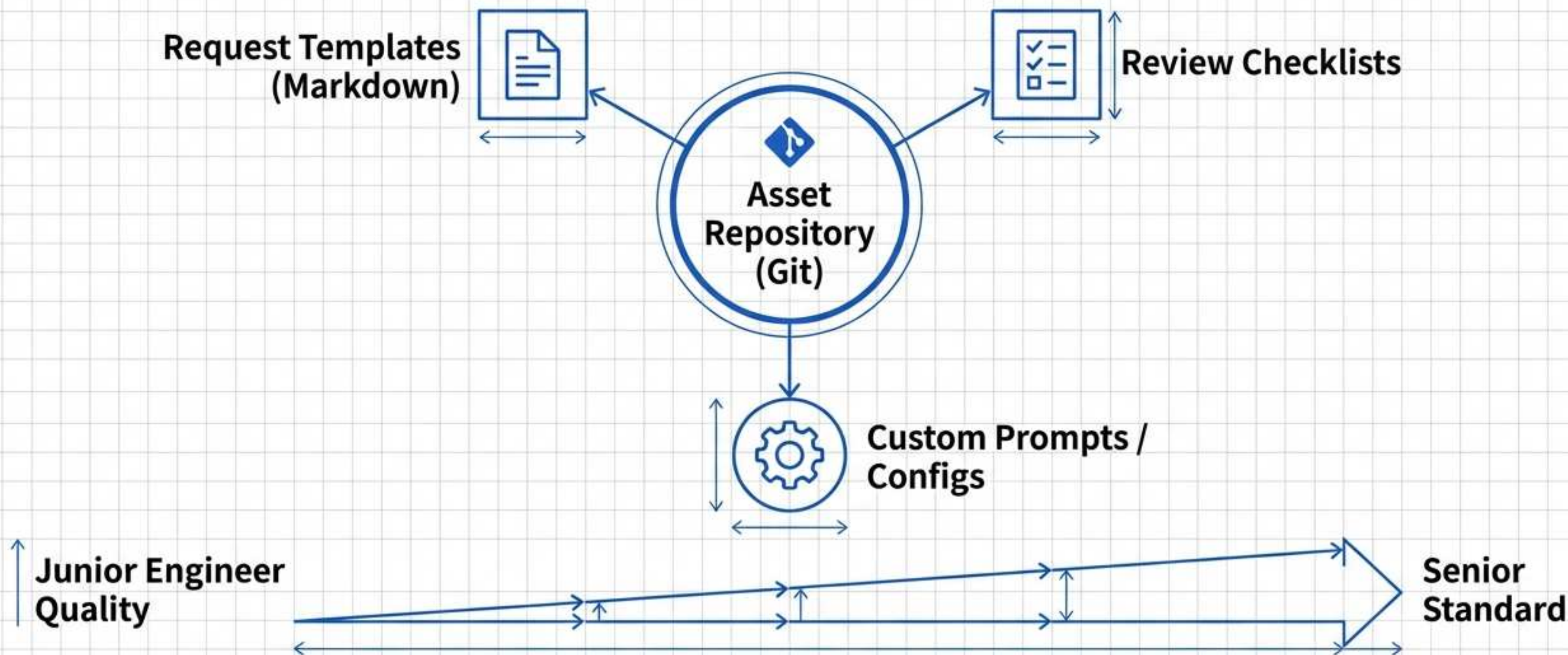
```
$ gh copilot suggest "Refactor this function to reduce complexity"  
> Suggesting...  
> Solution generated.  
> Copying to clipboard...  
$ cat prompt_history.log | grep "Refactor"
```

Automation

シェルスクリプトに
組み込み可能。

Noto Sans JP Bold: 「属人芸」を「チーム資産」に変える

個人のプロンプトテクニックをGit管理し、チーム全体の品質ボトムアップを図る。



モデル差分を吸収するレイヤー設計



Coreを標準化し、Interfaceのみを書き換えることで、モデル変更時の移植コストを最小化する。

セキュリティとガバナンスの絶対ルール



01. No Secrets

個人情報・顧客データ・APIキーは絶対に入力しない。

02. Masking

必要なデータは User_A のようにマスキング処理を行う。

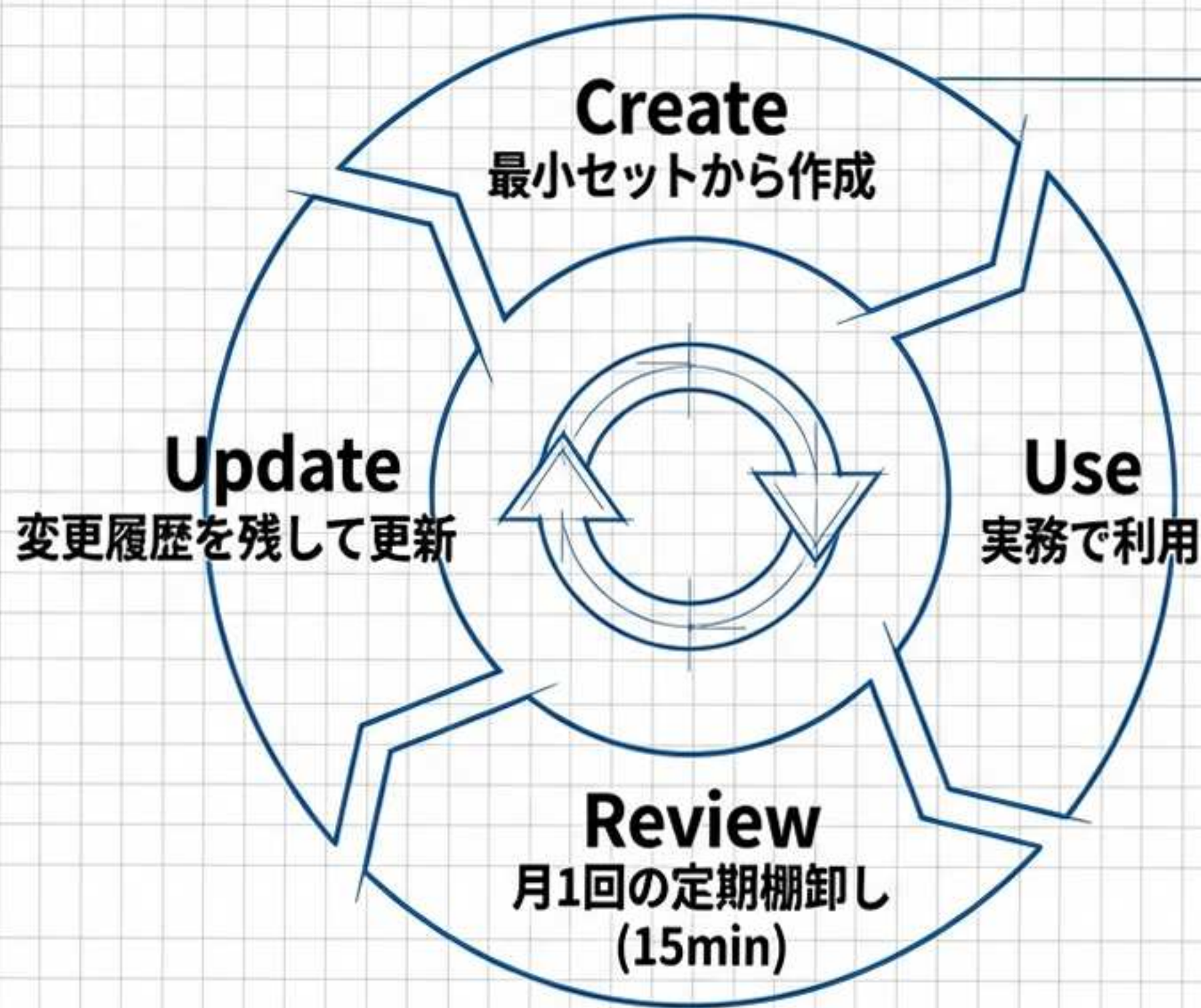
03. Log Awareness

学習利用のオプトアウト設定とログ保存期間を確認する。

04. Guidelines

「やってはいけないこと」を明文化し、チーム内で合意する。

資産の形骸化を防ぐメンテナンス運用

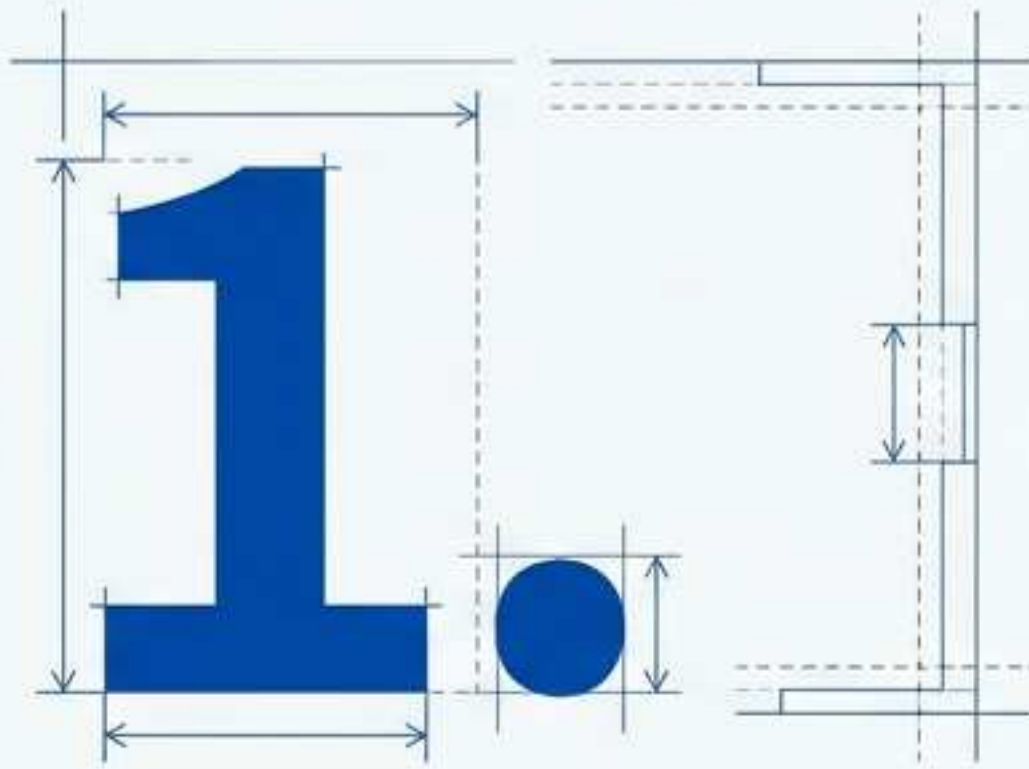


Troubleshooting

- ルールが多すぎる
→ まずはテンプレート1枚から
- 古くなる
→ 使われていないものは捨てる
- ブラックボックス化
→ Change Logを必須にする

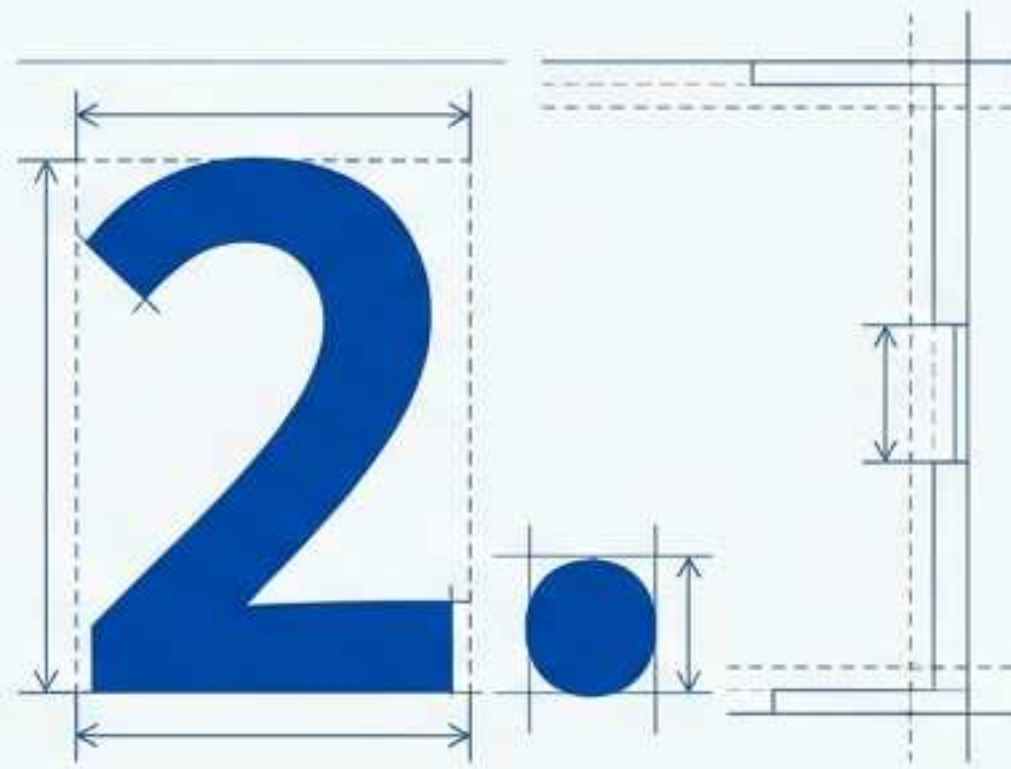
まず着手すべき「最小セット」

最初から完璧を目指さない。今日からできる3つのステップ。



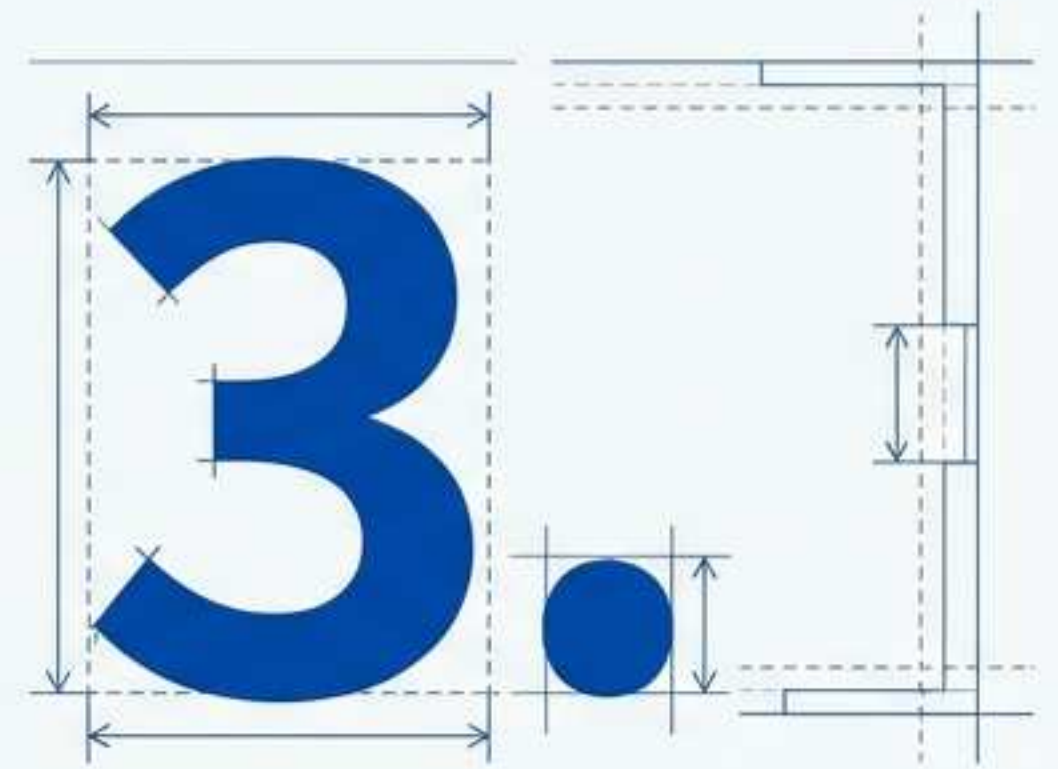
Template

よくある依頼のテンプレートを1枚作成する（Markdown形式）。



Split

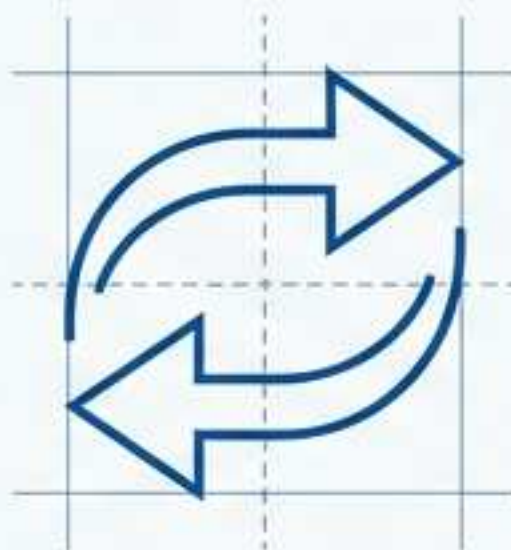
複雑なタスクを「調査・設計・実装」に分割して依頼する。



CLI History

一度だけCLIでAIを呼び出し、履歴が残る便利さを体験する。

ツール選びより「運用設計」が成果を決める



Interaction Turns
(やり取り回数)



Time per Task
(所要時間)



Kickback Count
(差し戻し数)

**AI活用はエンジニアリングである。
依頼を設計し、運用を自動化し、知見を資産化することで、
コストパフォーマンスは最大化される。**